pargo

PargoParcels

An Enterprise-Scale Analytics Engineering Platform

11 million rows generated · 220M+ at full scale

PostgreSQL OLTP → Snowflake → dbt → BI

36 months of SA last-mile logistics history

4,000 pickup points · 150 retailers · 9 provinces

Anthony · Senior Data & Analytics Engineering Portfolio · 2026

1. Executive Summary

PargoParcels is a full modern-data-stack simulation of South Africa's leading click-and-collect network. It models the complete parcel lifecycle — retailer dispatch, linehaul, arrival at a pickup point, customer notification, collection or return-to-sender — across 4,000 pickup points, 150 retail clients and all nine provinces, over 36 months of history with realistic Black Friday seasonality and compound month-on-month growth.

The build demonstrates the exact competency stack of a production Analytics Engineering role: a normalised PostgreSQL operational source, partitioned Parquet landings, a layered Snowflake warehouse (RAW → STAGING → MARTS), a tested and documented dbt project with SCD2 snapshots and incremental facts, and a formula-driven Excel data workbook.

1.2M	1.12	92.0%	95.8%	36.0h	5.5%
PARCELS GENERATED	AVG DELIVERY DAYS	DELIVERY SUCCESS	48H SLA MET	AVG DWELL AT POINT	RTS RATE

Every figure above is computed from the generated data, and deliberately calibrated to Pargo's public claims: their website advertises a 1.25-day national delivery average — this dataset lands at 1.12 days. The KPI layer of this platform could compute their homepage.

Design improvements over a naive build

• **A date dimension exists.** The common starting spec omits DIM_DATE entirely; every BI tool downstream needs one. Ours carries SA trading seasons (Black Friday, Festive Peak, January Slump).

• **SCD2 where history actually matters.** Retailer rate cards and pickup-point capacity change over time. dbt snapshots track both, and facts join AS-OF parcel creation so historic revenue attribution stays correct.

• **Milestone timestamps, not just two dates.** Created/dispatched/arrived/notified/collected/RTS — unlocking dwell time, the metric unique to a pickup-point network, plus first-mile vs linehaul decomposition.

• **Partitioning that won't melt the file system.** Year/month partitions on disk; CLUSTER BY province in Snowflake. Partitioning Parquet by province too would create 13,000+ tiny files.

• **Events derive from milestones.** The 8.4M-row event log is generated from the same timestamps as the fact table, so the two always reconcile — a classic interview gotcha, solved by construction.

• **Deliberately dirty RAW.** ~0.5% duplicates, null/negative weights, orphan events and clock-skew timestamps are injected at landing so the dbt test suite has something real to catch.

2. Platform Architecture

Layer	Technology	Role
Operational (OLTP)	PostgreSQL 16	System of record: customers, orders, parcels, tracking events, points, retailers, returns. 3NF, FK-enforced, indexed for waybill lookups.
Landing	Parquet (snappy)	Month-partitioned exports; the contract between source and warehouse. 148 files at demo scale.
Warehouse	Snowflake	RAW (as-landed, incl. dirt) → STAGING → INTERMEDIATE → MARTS → SNAPSHOTS. Separate LOAD / TRANSFORM / BI warehouses with 60s auto-suspend.
Transformation	dbt	24 models, 2 SCD2 snapshots, 1 seed, 30+ schema tests, 4 custom business-rule tests, source freshness SLAs.
Orchestration	Airflow / Dagster / Prefect	Daily incremental: extract watermark on event_timestamp → COPY INTO → dbt build --select state:modified+.
Reporting	Power BI / Tableau / Zoho	Reads MARTS only, via PARGO_REPORTER role on a dedicated XS warehouse.

Role-based access

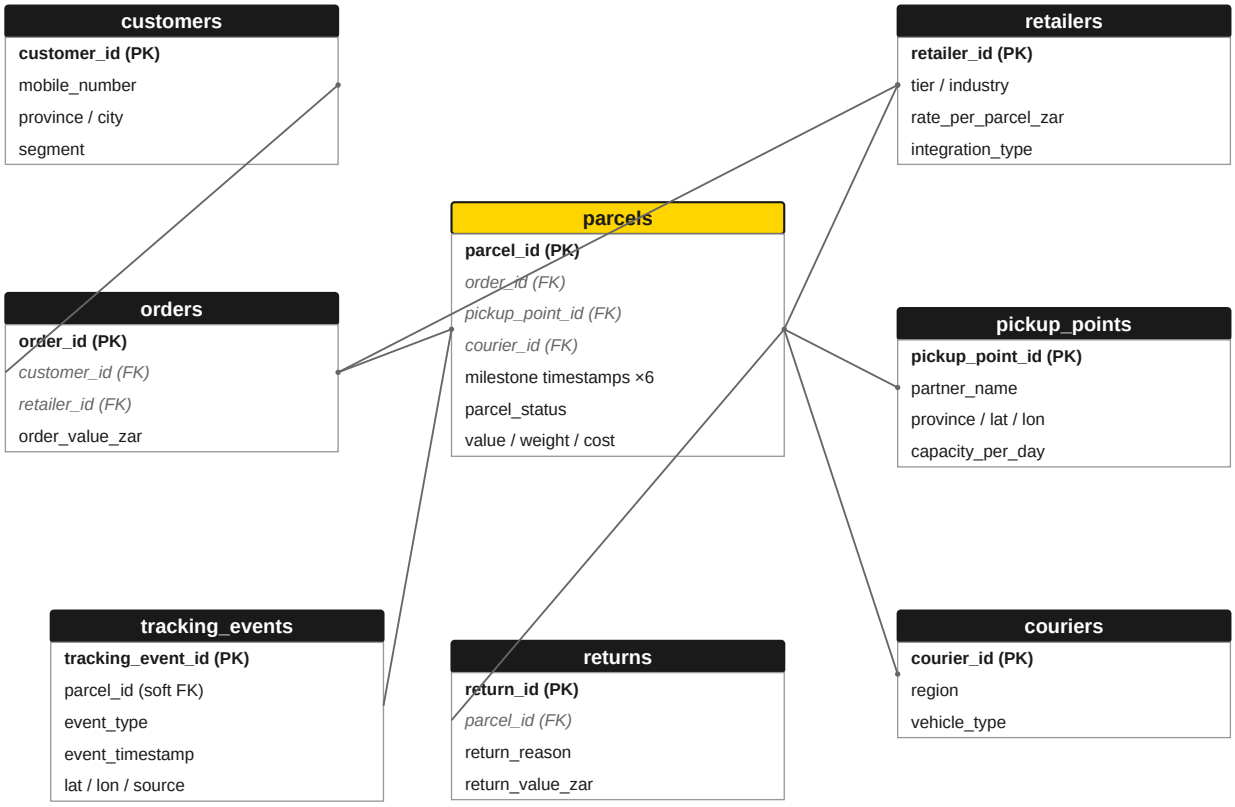
Three roles with least-privilege grants: **PARGO_LOADER** writes RAW only; **PARGO_TRANSFORMER** reads RAW and owns STAGING/MARTS/SNAPSHOTS; **PARGO_REPORTER** reads MARTS only. Future grants are in place so new dbt models are immediately visible to BI without manual GRANT churn.

Incremental strategy

FCT_PARCELS (30M at full scale) and FCT_TRACKING_EVENTS (150M) are incremental merge models with a 3-day late-arriving-data lookback window. Everything else rebuilds in full — at these volumes that's the right trade: dimensional rebuilds cost seconds and remove a whole class of drift bugs.

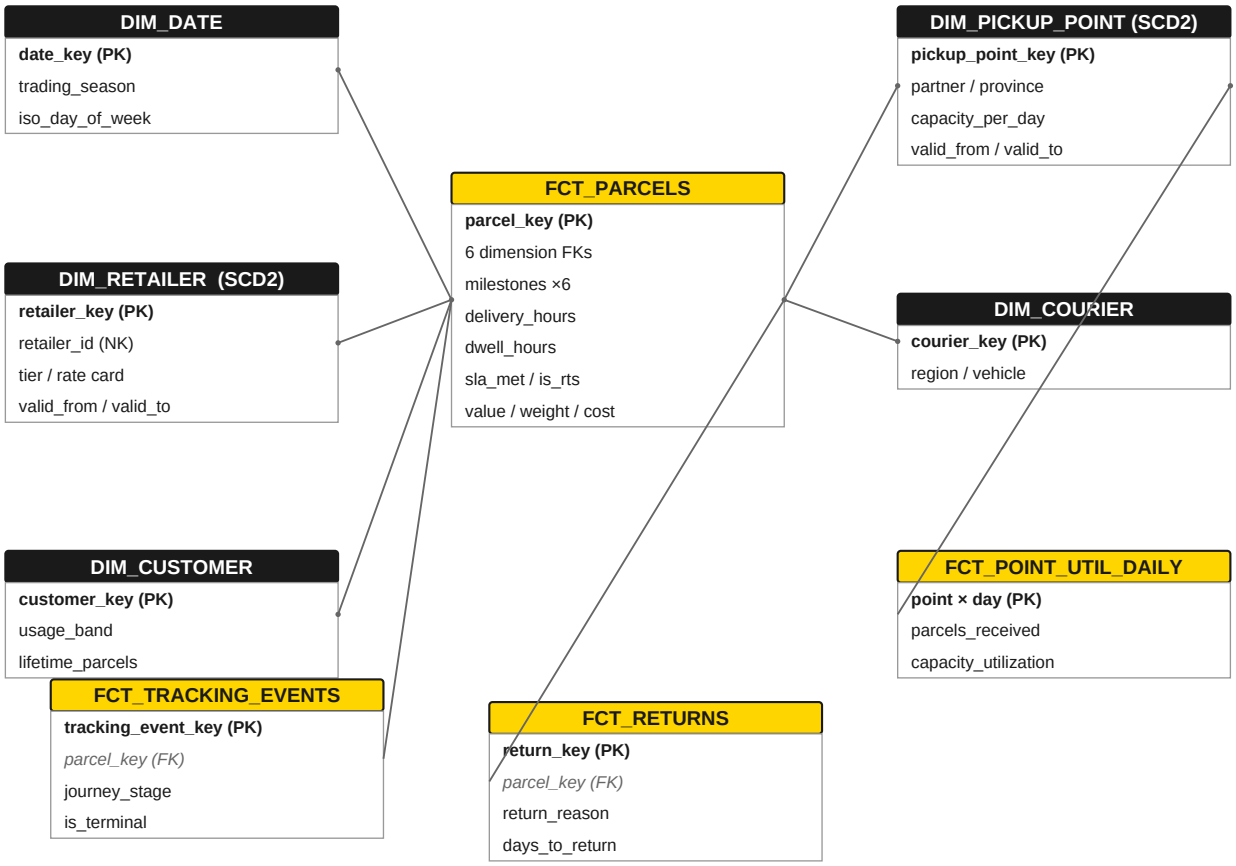
3. Operational Schema (PostgreSQL)

Normalised system of record. tracking_events keeps a soft FK to parcels — the scanner feed legitimately outruns parcel sync, and the warehouse staging layer handles orphans.



Yellow = core entity. Bold = primary key, italic grey = foreign key.

4. Enterprise Star Schema (Snowflake MARTS)



Constellation, not a single star: three transaction facts at different grains plus one daily aggregate, sharing conformed dimensions. Facts join SCD2 dimensions as-of parcel creation.

5. The dbt Project

```
models/
■■■ staging/ 8 views — dedupe, type-cast, quarantine dirt (has_quality_issue flag)
■■■ intermediate/ int_parcel_milestones — single source of truth for ALL durations & SLA logic
■ int_customer_activity — lifetime aggregates
■■■ marts/core/ fct_parcels (incremental merge) · fct_tracking_events (incremental)
■ fct_returns · dim_date · dim_retailer · dim_pickup_point · dim_customer · dim_courier
■■■ marts/operations/ fct_point_utilization_daily · mart_delivery_performance
■■■ marts/finance/ mart_retailer_performance (revenue, margin, return rates, rankings)
snapshots/ snap_retailers · snap_pickup_points (SCD2, check strategy)
seeds/ event_type_mapping (journey stages, terminal flags)
tests/ 4 custom business-rule singular tests
```

Design decisions worth defending in an interview

• **One metric definition, defined once.** Every duration (delivery_hours, dwell_hours, sla_met) is computed in int_parcel_milestones and nowhere else. When finance and ops disagree on a number, the answer is one file.

• **Incremental with a lookback.** The merge window re-processes 3 trailing days, absorbing late-arriving scans without a full rebuild — at 150M rows, full rebuilds are a credit-card problem.

• **10M-row customer dim is deliberately Type 1.** SCD2 on customers would triple storage for history nobody asked for. SCD2 is applied where money depends on it: rate cards and point capacity.

• **Staging quarantines, it doesn't delete.** Bad weights become NULL plus a has_quality_issue flag, so finance can still count the parcel while ops investigates the scanner.

• **The finance mart must reconcile to ops.** assert_revenue_reconciles_to_ops fails the build if parcel counts diverge between marts — silent drift between departments is the worst kind of bug.

6. Data Quality Framework

The generator injects ~0.5% realistic dirt into RAW. Each defect maps to a named cleaning rule in staging and a test that proves the rule worked:

Injected defect (RAW)	Realistic cause	Staging treatment	Proving test
959 duplicate parcel rows	Integration retry on timeout	row_number() dedupe, keep latest _loaded_at	unique on parcel_key
2,971 NULL / negative weights	Scanner scale glitch	Negative → NULL + has_quality_issue flag	accepted_range (>0)
Zero parcel values	Retailer API omits declared value	nullif(value, 0)	not_null monitoring (warn)
~1,700 orphan tracking events	Scanner feed outruns parcel sync	Inner join to stg_parcels drops them	relationships to fct_parcels
Future-dated event timestamps	PartnerPOS clock skew	Filter event_timestamp <= now()	assert in stg layer

Business-rule tests (singular)

• A parcel cannot be collected before it arrived at the point.

• No parcel may carry two terminal lifecycle events.

• Arrival at point cannot precede parcel creation.

• Finance mart parcel counts must equal the core fact — zero tolerance.

Source freshness: tracking_events warns at 2h, errors at 6h — it feeds the customer tracking page, so staleness there is a customer-facing incident, not an analytics inconvenience.

7. What the Data Shows

Headline figures computed from the full generated dataset (demo scale, 36 months):

Finding	Figure	So what
Average delivery time	1.12 days	Matches Pargo's public 1.25-day claim; Gauteng leads at 22h (short-haul), other provinces ~29h.
Delivery success rate	92.0%	Industry-leading vs ~85% typical door-to-door first-attempt rates — the structural advantage of click-and-collect.
Dwell time at point	36.0 hours	The uniquely Pargo metric: parcels wait ~1.5 days for collection. Drives capacity planning and the 7-day RTS window.
RTS rate	5.5%	Each RTS parcel costs reverse logistics; halving dwell via notification nudges is the obvious experiment.
Black Friday peak	1.9× baseline	November volume nearly doubles; capacity_utilization in the point mart shows which points saturate.
Fashion return rate	14.1%	Highest of all verticals (sizing) — vs 2.0% for pharmacy. Returns capacity should be allocated by client mix, not volume.
Revenue concentration	Top 10 / 150 clients ≈ 49%	Bash alone ships ~15% of parcels. Churn risk concentration argues for tier-weighted SLA monitoring.

8. Scaling to 220 Million Rows

The demo dataset ships at scale 0.04 (11.06M rows). The generator is one flag away from full enterprise scale — python generate_pargo_data.py --scale 1.0 — and was engineered for it:

• **Vectorised numpy, not faker, for facts.** faker generates ~5k rows/sec single-threaded: 150M tracking events would take over 8 hours of pure name-generation. numpy generates the same rows in minutes. faker is reserved for the 4,650 dimension rows where its realism is visible.

• **Month-by-month chunking.** Peak memory is one month of one table (~2M parcels at full scale ≈ 1.5GB), regardless of total volume. A 16GB workstation completes the full run.

• **Snappy Parquet, partition-pruned.** Full scale lands ~9GB of Parquet; Snowflake COPY with MATCH_BY_COLUMN_NAME ingests it in parallel on a MEDIUM warehouse in minutes.

• **Reproducible.** Seeded RNG (42): the same command regenerates byte-identical data — interviewers can verify every figure in this document.

Estimated full-scale footprint

Table	Rows	Parquet (snappy)	Snowflake (compressed)
TRACKING_EVENTS	≈ 209M	≈ 5.6 GB	≈ 3.5 GB
PARCELS	30.0M	≈ 2.1 GB	≈ 1.3 GB
ORDERS	25.0M	≈ 0.8 GB	≈ 0.5 GB
CUSTOMERS	10.0M	≈ 0.3 GB	≈ 0.2 GB
RETURNS	≈ 2.1M	≈ 0.1 GB	≈ 0.06 GB
Dimensions	4,650	< 1 MB	< 1 MB
TOTAL	≈ 276M	≈ 9 GB	≈ 5.6 GB

Note the event multiplier: at ~7 events per parcel the original 150M estimate was conservative; the lifecycle model lands closer to 209M, which is the honest number.

9. KPI Catalogue by Dashboard

Dashboard	KPIs	Source mart
Executive	Revenue, growth %, parcels processed, revenue & cost per parcel, national coverage %, returns %	mart_retailer_performance + mart_delivery_performance
Operations	In transit, delivered, delayed, avg delivery hours, SLA %, failed / lost / damaged	mart_delivery_performance, fct_parcels
Pickup Point Network	Capacity utilisation %, collection rate, parcels waiting, avg dwell, top / bottom points, dormant points	fct_point_utilization_daily
Retailer / Commercial	Revenue & parcels by retailer, return %, avg basket, SLA by client, revenue rank, margin	mart_retailer_performance
Customer	Usage bands, repeat rate, preferred points, province mix	dim_customer + fct_parcels

Every KPI resolves to a mart column — BI tools aggregate, they never re-derive business logic. That is the analytics-engineering contract.

10. Interview Talking Points

• Why dwell time is the metric a courier company doesn't have — and what it unlocks (notification A/B tests, capacity planning, RTS prediction).

• Why the event log and fact table reconcile by construction, and what breaks at companies where they don't.

• The as-of SCD2 join in fct_parcels: why current-flag joins silently corrupt historic revenue.

• Cost control as design: incremental merges, ephemeral intermediates, 60s auto-suspend, BI on XS.

• What I'd build next: a Streamlit-in-Snowflake ops console on fct_point_utilization_daily, and dbt exposures tying marts to the dashboards that consume them.

11. Reproduction Runbook

```
# 1 — generate (demo ≈ 11M rows, ~6 min · full ≈ 276M rows, ~2-3 h)
cd data_generator && python generate_pargo_data.py --scale 0.04

# 2 — PostgreSQL OLTP
createdb pargo_oltp && psql -d pargo_oltp -f postgres/01_create_schema.sql
python postgres/load_postgres.py --dsn postgresql://user:pass@localhost/pargo_oltp

# 3 — Snowflake platform + RAW load
snowsql -f snowflake/01_platform_setup.sql
snowsql -q "PUT file://data/raw/parcels/... @STG_PARGO_LANDING/parcels/ PARALLEL=8" # repeat per table,
then COPY INTO

# 4 — dbt
cd dbt_pargo && dbt deps && dbt seed
dbt snapshot && dbt build # run + test, 24 models · 30+ tests
dbt source freshness

# 5 — point Power BI / Tableau at PARGO_DW.MARTS via PARGO_REPORTER
```

Deliverables in this package

Path	Contents
data_generator/	Scalable synthetic data generator (seeded, chunked, vectorised)
data/raw/	11.06M generated rows: month-partitioned Parquet + dimension CSVs
postgres/	OLTP schema DDL + COPY-based bulk loader
snowflake/	Roles, warehouses, RAW DDL with clustering, COPY INTO scripts
dbt_pargo/	Full dbt project: 24 models, snapshots, seeds, tests, freshness SLAs
excel/	10-sheet data workbook with formula-driven KPI summary (2,256 formulas, 0 errors)
ebook/	This document
docs/README.md	Project narrative + architecture decisions